

FPGA Acceleration of Neural A* Search: Profiling, Quantization, and Hardware Cost

Moamen Kassem [Github Link](#)

02360509 — *Advanced Topics in Hardware Accelerators for Deep Learning*

ABSTRACT

This project accelerates Neural A* inference on a PYNQ-Z2 board carrying a Xilinx Zynq-7020 FPGA. Profiling showed that the convolutional encoder accounts for 92-95% of inference time, so the accelerator targets the encoder rather than the search, and the Bloom filter proposed for the search's visited-set test proved unnecessary on a grid, where an exact bit-vector is smaller and faster. The encoder distributed with the reference implementation requires 3.47 times the LUTs the device provides. A 102-times-smaller encoder, trained by distillation with 4-bit weights, occupies 66.3% of the device, meets timing, and computes a guidance map in 3.91 ms on the board against 6.70 ms for the published encoder on a CPU, planning within 0.01 of its own software model on every metric. Two further results concern measurement: an unfolded output quantizer cost 174 times the accelerator's own latency until the terminal sigmoid was moved to the processor, and the reduction in generated nodes, which the expansion metric does not count, is about half the reduction in expanded nodes.

1. INTRODUCTION

This project accelerates the inference stage of Neural A* ^[1], a path planner that combines a convolutional encoder with classical A* search ^[6], on a PYNQ-Z2 board carrying a Xilinx Zynq-7020 FPGA. The original proposal was to accelerate the search itself and to replace its visited-set test with a counting Bloom filter; profiling redirected the work to the encoder, and the sections below give the measurements behind that decision and the accelerator that followed.

2. BACKGROUND

2.1 Neural A* search

A* ^[6] expands nodes in order of $f(v) = g(v) + h(v)$; the reference implementation computes $0.5 \cdot (g + h)$, which gives the same order. The term $g(v)$ is the cost of the best-known path from the start to v : every step from a cell v into a neighboring cell v' adds that cell's movement cost $c(v')$, so $g(v') = g(v) + c(v')$. The term $h(v)$ estimates the remaining cost to the goal; on an 8-connected grid the standard choice is the Chebyshev distance. In vanilla A* the movement cost is uniform, $c(v') = 1$ for every free cell, so the search is guided by geometry alone.

Neural A* keeps h unchanged and replaces $c(v')$ with a learned value $\phi(v')$ between 0 and 1 from the guidance map, so that $g(v') = g(v) + \phi(v')$. Cells the network expects to lie on the optimal route become cheap and are explored first; cells in dead ends become expensive and are explored late or not at all. The network runs once per problem, before the search begins, and is never consulted during the search.

The network that produces ϕ is called the encoder. Its input is two 32×32 channels: the obstacle map, and a map marking the start and goal cells. Each layer is a 3×3 convolution followed by batch normalization and a ReLU; the final layer reduces to a single channel and applies a sigmoid, giving one value in $(0, 1)$ per cell. No layer changes the spatial resolution, so the output is again a 32×32 map, and each value depends on the neighborhood around its cell — 9×9 cells for the four-layer encoders used in this work.

The reference implementation contains two versions of the search. DifferentiableAstar is a dense tensor formulation used only during training: every step performs a softmax over the entire map, so a full search costs

$O(N^2)$ operations for $N = H \times W$ cells — about a million for a 32×32 grid — but gradients can flow through it. `pq_astar` is an ordinary priority-queue A* used at inference. Only the latter is a sensible target for hardware, and it is the version implemented here.

2.2 Metrics

Planning quality is measured as in the Neural A* paper^[1]. `p_opt` is the fraction of problems on which Neural A* returns a path as short as the optimal one. `p_exp` is the average over problems of the reduction in expanded nodes relative to vanilla A* on the same problem. Section 5 adds `p_gen`, the same reduction measured over generated nodes.

2.3 Target platform and toolchain

All hardware results are on a Digilent PYNQ-Z2 carrying a Xilinx Zynq-7020 (xc7z020clg400-1): 53,200 LUTs, 106,400 flip-flops, 140 BRAM-36K blocks and 220 DSP slices, alongside a dual-core ARM Cortex-A9 processor at 650 MHz. Quantized models are expressed in Brevitas^[8] and compiled with FINN^[2], which builds a dataflow accelerator in which every layer has its own hardware, all layers run concurrently, and weights are held on chip.

Two details of how FINN builds an accelerator matter later. Each convolution becomes a matrix-vector unit whose parallelism is set by two numbers, SIMD and PE; their product is the layer's multiply-accumulates per cycle and choosing them for every layer is called folding. More per cycle means fewer cycles per frame and more area. Batch normalization, activation and quantization after a convolution are not computed at run time: FINN converts them into threshold comparisons, so the hardware compares the convolution's integer result against precomputed boundaries and reads off the quantized output directly.

Software timings are reported on two processors: an x86 server, and the board's own ARM processor. All latencies are for a single problem at a time, which is how a planner is used.

3. REFERENCE IMPLEMENTATION

Every speedup in this report is measured against a software baseline, so the baseline must be sound. I wrote my own implementation of the `pq_astar` algorithm, preserving its semantics exactly while changing the data structures. The neighbor table and the heuristic map are precomputed once per problem as arrays, where the reference derives both per node during the search. The closed list, which the reference stores in a `pqdict` priority queue, becomes a boolean array with one entry per cell. The open list uses the standard library heap. Upstream stores only `f` and updates it incrementally; I store `g` explicitly and compute `f` from it, which produces the same order of expansions.

Validation was in two directions. Against upstream, my implementation produces identical path lengths and expansion counts on 450 test problems with real guidance maps. On unit-cost maps it reproduces the dataset's own shortest-path distances exactly.

One difference between the two upstream searches matters for Neural A*. `DifferentiableAstar` adds the guidance map to both `g` and `h`, whereas `pq_astar` adds it only to `g`, as in the paper's equations. The upstream tests use only vanilla A*, where the guidance map is all ones, so the difference is never exercised. To check whether following `pq_astar` matters, I ran the search both ways with the same guidance maps: adding `φ` to `g` only gives `p_opt` 0.853 and `p_exp` 0.449, and adding it to both gives 0.823 and 0.457. The difference is small, so the choice does not affect the results below.

Storing `g` explicitly also suits hardware, because a running sum of values below 1 has a bounded, predictable range. Sizing the search for a hardware implementation, the largest `f` observed over the evaluation set was 16.02,

and a bit-accurate simulation showed that ten fractional bits reproduce the floating-point search exactly, while six cost about 1% extra expansions. The resulting format is `ap_fixed<17,7>`, the Vitis HLS fixed-point type with 17 bits in total, of which 7 hold the integer part including the sign and 10 the fraction: it covers -64 to just under $+64$ in steps of $1/1024$. The largest open list observed held 123 entries, so the whole priority queue fits in 412 bytes. The engine was sized but not built.

All speedups are measured against this baseline rather than against published timings. I make no controlled comparison with the runtime table in ^[1], because neither the hardware nor the sampling of start and goal positions is matched.

4. WHERE THE TIME GOES

Table 1 shows how inference time divides between the two stages.

Map size	CNN encoder	A* search	Encoder share
32×32	6.70 ms	0.60 ms	92%
64×64	22.26 ms	1.20 ms	95%

Table 1: Single-problem inference time on the x86 host, by stage.

The encoder dominates, and its share grows with map size. By Amdahl's law, accelerating only the search — even to zero time — would speed up the whole pipeline by at most $1/0.92 = 1.09$ times on 32×32 maps, whereas accelerating the encoder tenfold would give 5.8 times. Figure 1 shows both curves. The accelerator in this project therefore targets the encoder.

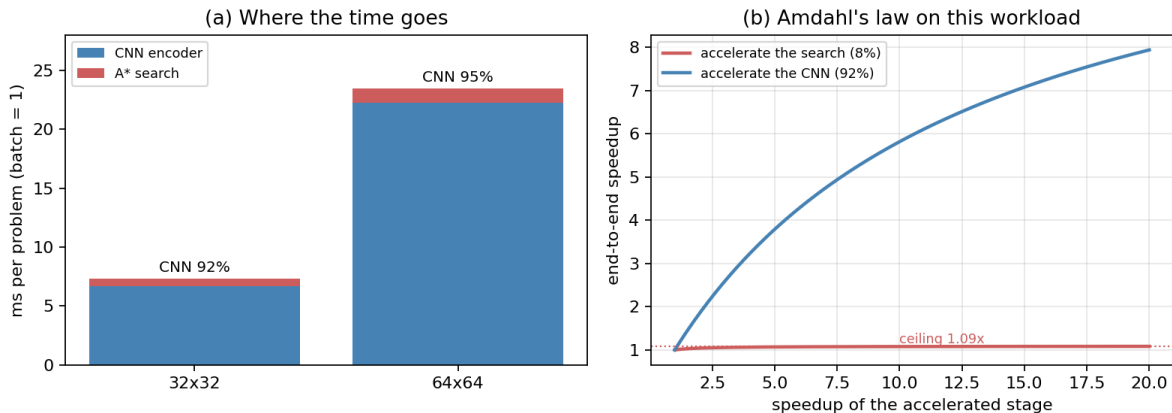


Figure 1: (a) Measured time per stage at two map sizes. (b) End-to-end speedup as a function of the speedup achieved on each stage.

5. EXPANDED AND GENERATED NODES

The Neural A* paper measures how well the network focuses the search by the reduction in expanded nodes — nodes taken from the open list and processed — and its training loss targets the same quantity. That is the natural measure of search effort in software. Every generated node, meaning every neighbor inserted into the open list, also costs work in any implementation: a guidance-map read, a heuristic read, an f computation and a queue insertion, whether or not it is later expanded. That work is not counted by the expansion metric, so I also measured `p_gen`, the reduction in generated nodes, on the same problems. Table 2 compares the two.

Metric	Reduction vs vanilla A*	Problems where Neural A* is worse
Expanded nodes (p_exp)	44.9%	11.0%
Generated nodes (p_gen)	24.9%	21.7%

Table 2: Expanded and generated nodes relative to vanilla A*, float encoder, 300 problems.

The reduction in generated nodes is about half the reduction in expansions. Each expanded node still generates its unexplored neighbors, so a search that expands fewer nodes does not generate proportionally fewer, and on about one problem in five Neural A* generates more nodes than vanilla A* while expanding fewer. The ratio holds across every encoder in Table 5 (Figure 2b), and on the board the deployed configuration reduces expansions by 41.5% and generated nodes by 23.4%. The paper’s definition counts a problem where Neural A* expands more nodes than vanilla A* as a zero reduction; counting those as negative gives 43.6% rather than 44.9%. Both metrics are valid and answer different questions; for sizing an implementation, generated nodes are the relevant one.

6. THE VISITED-SET TEST

The proposal’s second element was a counting Bloom filter^[7] for the search’s visited-set test, following Falcon^[3], whose query pipelines track visited nodes with on-chip Bloom filters rather than a visited array or a hash table. A Bloom filter trades exactness for memory: it can report an unvisited node as visited, but it needs far less storage than an exact record of every node. Whether that trade is worth taking depends on how many nodes there are and how their identifiers are distributed. On the graphs Falcon targets it plainly is: with ten million nodes and 32-bit identifiers, an exact record of the visited set would not fit in on-chip memory at all, whereas a 256 Kbit filter answers the same question with about one false positive in 600,000.

Profiling the search on this problem showed that neither condition holds, which is what made a better option available. On a grid every node is addressed by its index $\text{row} \times W + \text{col}$, so identifiers are dense integers below 1024 for a 32×32 map, and the peak open list across the test set held 123 entries. The visited set can therefore be stored as one bit per cell — the representation my implementation uses (Section 3), and in hardware a 1024-bit vector: one block RAM, one clock cycle per lookup, exact, and 0.02% of the device. A Bloom filter at this scale would need several hash computations and memory accesses per lookup, occupy more memory, and still return false positives, which in A* cause an unvisited node to be skipped and the path to get worse. The identifier space and the node space coincide on a grid, so exactness costs nothing and there is nothing left for the filter to buy.

This also differs from the reference implementation, which stores its closed list in a pqdict — the same priority-queue structure it uses for the open list — although the closed list is only ever asked whether a node is in it, and nothing is ever taken from it in order. Every insertion there maintains a heap ordering that is never used. The only structure in the search that genuinely needs ordering is the open list, and at 123 entries it is small.

7. QUANTIZATION

FINN requires a quantized network. Simply rounding the trained float weights, known as post-training quantization, damages it: at W4A8 (4-bit weights, 8-bit activations) p_opt falls from 0.830 to 0.610. The network must be retrained with quantization in place so that its weights adapt to the rounding. Retraining through the full Neural A* objective is expensive because it runs the differentiable search, so I trained by distillation instead: the already trained float encoder produces a guidance map for each training problem, and the quantized network, starting from random weights, is trained to reproduce those maps. Since the search uses nothing but the guidance map, a network that reproduces the float maps reproduces the float searches. Table 3 shows the result.

Configuration	p_opt	p_exp	Weights	% of BRAM
float32	0.830	0.454	1560 KB	248%
W8A8	0.840	0.457	381 KB	60%
W4A8	0.845	0.455	190 KB	30%
W4A4	0.800	0.435	190 KB	30%
W2A8	0.545	0.398	95 KB	15%
W2A4	0.600	0.391	95 KB	15%
W2A2	0.710	0.335	95 KB	15%

Table 3: The published 391k-parameter encoder at different precisions, quantized rows trained by distillation, 200 test problems.

W4A8 costs nothing measurable relative to float32 — the small differences are within sampling noise — while halving the weight memory of W8A8. Below four bits of weight, quality drops sharply. p_opt behaves non-monotonically there, W2A2 scoring higher than W2A8, but this is not an improvement: as precision falls the guidance map flattens toward uniform, and a uniform map is exactly vanilla A*, which always finds optimal paths while saving no expansions. p_exp falls steadily over the same rows, from 0.398 to 0.335, confirming that the network is losing its guidance.

8. WHY THE PUBLISHED ENCODER CANNOT BE DEPLOYED

The encoder distributed with the reference implementation has five 3×3 convolutions with channel widths 2-32-64-128-256-1: 391,395 parameters and 399 million multiply-accumulates per 32×32 map. I built it for the Zynq-7020 at W4A8 and 100 MHz with 342 MACs per cycle, the folding needed for about 11.8 ms per frame — already slower than the 6.70 ms software baseline. Vivado rejected the design during design-rule checking, before placement began; Table 4 gives the shortfall.

Resource	Required	Available	Over by
Slice LUTs	184,839	53,200	3.47×
LUT as logic	173,327	53,200	3.26×
Slice registers	182,164	106,400	1.71×
CARRY4	23,965	13,300	1.80×

Table 4: Design-rule failure for the published encoder. Vivado reported 'Error(s) found during DRC. Placer not run.'

Matching the software baseline would need roughly 600 MACs per cycle, about 1.8 times the folding that already exceeds the device by 3.47 times — on the order of six times the logic a Zynq-7020 provides. The published encoder cannot be accelerated on this device by changing the folding.

9. A RIGHT-SIZED ENCODER

A 32×32 maze does not obviously need 391,395 parameters to produce one value per cell. I trained narrower four-layer encoders by distillation from the float model and evaluated each for planning quality and hardware cost. Encoders are named by their three hidden layers: 8/16/16 has 8, 16 and 16 channels. Table 5 gives the results.

Encoder	Bits	Params	p_opt	p_exp	p_gen	MAC/cy	Thru. ms	LUT %
32/64/128/256	fp32	391,395	0.800	0.441	0.239	342	11.80	347 (m)
16/32/64	W4A8	24,135	0.730	0.407	0.226	50	5.90	61
16/32/32	W4A8	14,567	0.700	0.401	0.224	51	2.95	62
8/16/32	W4A8	6,311	0.715	0.396	0.218	52	1.47	62
8/16/16	W4A8	3,831	0.640	0.385	0.221	27	1.47	45 (m)
8/16/16	W4A8	3,831	0.675	0.364	0.206	54	0.74	64 (m)
4/8/8	W4A8	1,055	0.580	0.344	0.198	30	0.37	47
8/16/16	W2A8	3,831	0.545	0.316	0.184	27	1.47	45 (m)

Table 5: Encoder width and precision against planning quality and hardware cost. Throughput is FINN's cycle estimate at 100 MHz. Rows marked (m) are measured after synthesis; other LUT figures are projected. The emphasized row is the deployed architecture.

LUT figures for unbuilt rows use $\text{LUT} = 14,488 + 358 \times (\text{MACs per cycle})$, fitted to the two measured small designs: 24,154 LUTs at 27 MACs per cycle and 33,820 at 54. The constant is the part of the design that does not scale with parallelism — the Zynq interface, the DMA engines and the buffers between layers.

Each row was trained separately, and separate runs of the same architecture differ by about 0.02 in p_exp — the two 8/16/16 W4A8 rows are the same network trained twice — so differences below about 0.03 between rows should not be read as real.

Reducing the encoder from 391k to 3.8k parameters, a factor of 102, lowers p_exp from 0.441 to 0.407 as deployed — under eight percent. The published encoder is heavily over-provisioned for grids of this size.

At identical architecture, folding and LUT cost, W4A8 achieves p_exp 0.385 against 0.316 for W2A8, because LUT cost follows MACs per cycle rather than bit width. Four bits is the floor for this workload.

The wider encoders score slightly higher, but within the run-to-run variation: 16/32/64 reaches 0.407, the same as the deployed 8/16/16 weights. They are also slower, because more channels mean more work per cell at the same parallelism. At the folding in Table 5, 16/32/64 needs 5.90 ms per frame in throughput and about 22 ms for a single frame — slower than even the published encoder on the x86 server. 8/16/16 matches its planning quality at a fraction of the latency.

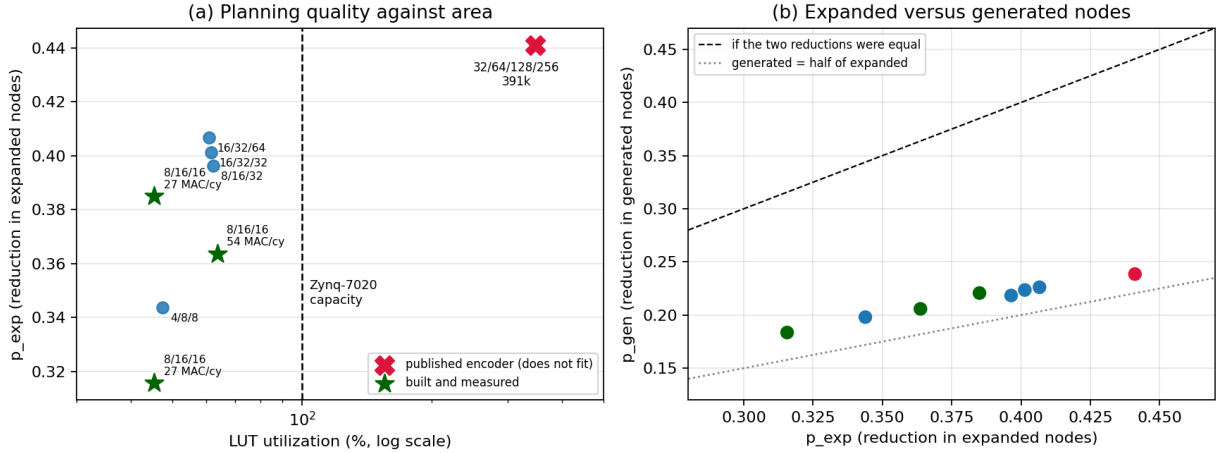


Figure 2: (a) Reduction in expanded nodes against LUT utilization; the published encoder lies far beyond the device’s capacity. (b) Reduction in expanded nodes against reduction in generated nodes for every configuration, discussed in Section 5.

10. HARDWARE RESULTS

10.1 The four builds

I deployed the 8/16/16 W4A8 encoder to the PYNQ-Z2 in four builds, each changing one thing:

- Build A, 27 MACs per cycle: the first working design, with the most conservative folding.
- Build B, 54 MACs per cycle with the same trained weights: doubles the parallelism to measure the trade-off between speed and area. Since the weights are unchanged, every difference from build A is due to the hardware.
- Build C, as build B with the final sigmoid moved from the accelerator to the ARM processor: fixes the output-format problem described in Section 10.3. This required retraining.
- Build D, as build C with a change to the training: tests whether that retraining can recover the path quality lost in build C. It could not (Section 10.3).

Table 6 gives their resource use.

Build	Folding	LUT	FF	BRAM-36K	DSP	Timing
A	27 MAC/cy	24,154 (45.4%)	26,992 (25.4%)	3 (2.1%)	30 (13.6%)	met
B	54 MAC/cy	33,820 (63.6%)	31,058 (29.2%)	5 (3.6%)	60 (27.3%)	met
C	54 MAC/cy, host sigmoid	35,266 (66.3%)	29,580 (27.8%)	5 (3.6%)	60 (27.3%)	met
D	as C, clipped targets	35,369 (66.5%)	29,902 (28.1%)	5 (3.6%)	60 (27.3%)	met

Table 6: Post-synthesis utilization. Build B met timing with 0.596 ns of slack against a 10 ns constraint, an achieved clock of about 106 MHz.

Build A takes 5.58 ms per frame; build B halves the cycle count to 3.00 ms, for 18 percentage points more LUT and twice the DSP slices. Builds C and D have the same folding as B and nearly the same resources. Block RAM stays almost unused — the 3,744 four-bit weights occupy 1.9 KB — so compute, not memory, limits this workload. FINN’s own resource estimate for build A was 4,502 LUTs against 24,154 measured, 5.4 times too low: useful for comparing configurations, not for deciding whether one fits. Raising the folding beyond 54 MACs per cycle gives no further speedup for this architecture, because SIMD must divide each layer’s input channels

and PE its output channels; the next useful step needs about 104 MACs per cycle, which the LUT model places at 97% of the device.

Each build's output was checked against the trained model's guidance maps on 64 held-out inputs; for build C the correlation is 0.962 and the mean absolute error 0.046.

10.2 Planning quality on the board

Table 7 compares vanilla A* and Neural A* with guidance maps computed by the accelerator, with the search running on the ARM processor. Figure 3 shows one instance end to end, alongside the published encoder on the same problem: vanilla A* generates 127 nodes and expands 70, the published encoder 85 and 22, and the deployed encoder on the board 75 and 22. On this instance the smaller encoder generates fewer nodes than the published one, although across the test set it is slightly behind (Section 9).

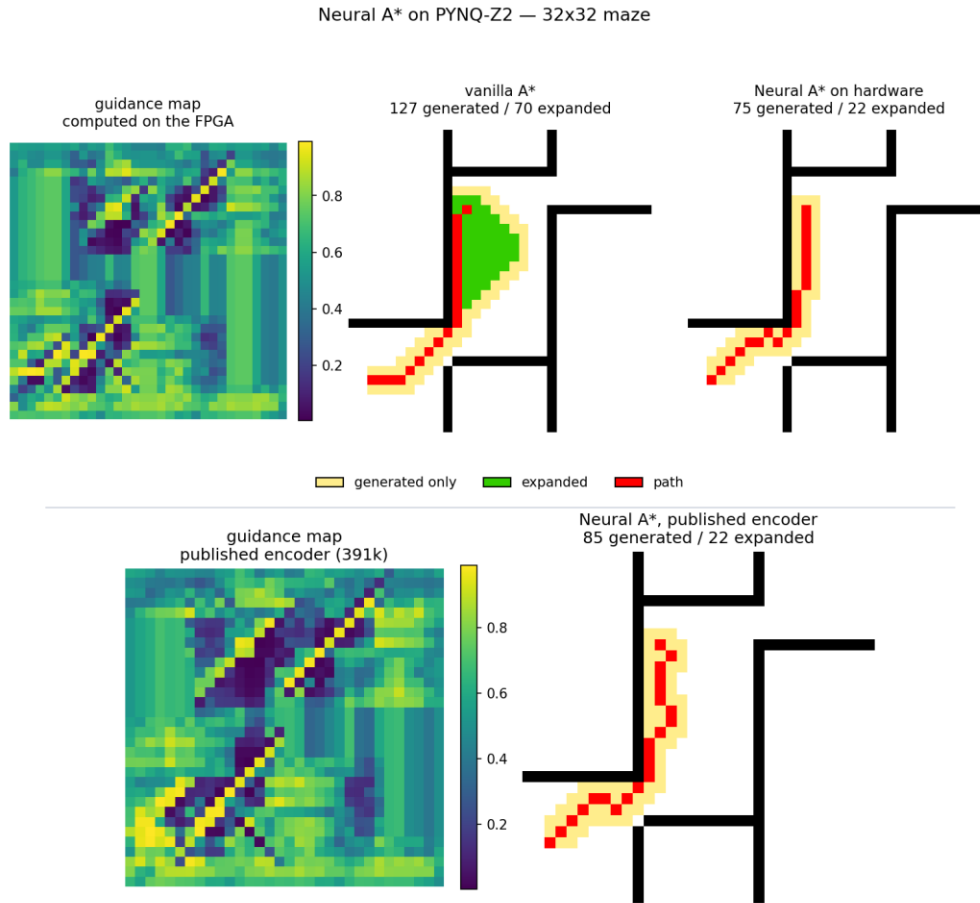


Figure 3: One problem solved three ways. Top: the guidance map computed by the accelerator, then vanilla A* and Neural A* with that map, both on the board. Bottom: the published 391k encoder on the same problem, in software. Pale cells were generated but never expanded; green cells were expanded; red marks the returned path.

Neural A* on hardware optimal
path with guidance map visual

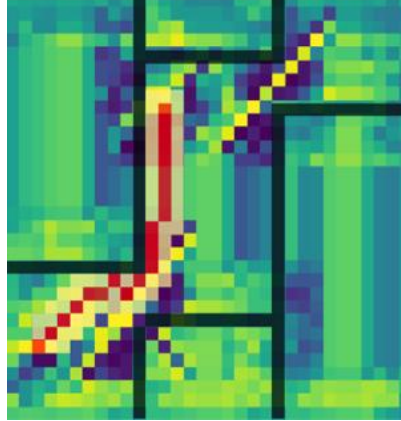


Figure 4: The search of Figure 3 overlaid on the guidance map computed by the accelerator. Dark cells are cheap, bright cells expensive; the returned path follows the cheap corridor.

Planner	Expanded	Generated	Path length	Encode	Search
Vanilla A*	135	200	29.9	—	33.07 ms
Neural A* (build C)	68	146	30.8	5.00 ms	21.88 ms

Table 7: Averages over 200 problems on the board, build C, with the search on the ARM processor. Encode is the whole per-problem cost of producing a guidance map: building the two input channels, the accelerator call, and the sigmoid on the processor. The accelerator call alone is 3.91 ms (Table 9).

These are the same 200 problems used to evaluate build C in software (Table 10, row C), so Table 8 compares the two directly.

Build C, same 200 problems	p_opt	p_exp	p_gen
Software (PyTorch model)	0.655	0.412	0.228
On the board (FPGA guidance maps)	0.665	0.415	0.234

Table 8: Planning quality of build C in software and on the board.

The board reproduces the software result to within 0.01 on every metric. The guidance maps computed by the accelerator are not bit-identical to the software model's — their correlation is 0.962 — but the differences are too small to change which nodes the search expands. The whole pipeline, from the quantized network through FINN and the bitstream to the search on the processor, therefore plans exactly as the trained model does.

10.3 Latency and the output format

Build B's accelerator needs 3.00 ms per frame, but the same build measured on the board took 521.91 ms — 174 times longer. The time was not spent in the accelerator but in reading its output.

The encoder ends with a convolution, batch normalization, a sigmoid and an output quantizer. FINN converts batch normalization and quantization into threshold comparisons (Section 2.3), but it could not do so with a sigmoid between them. It therefore stopped at the convolution and returned its raw result: the accumulator, the sum of all products for each cell, which needs 24 bits to hold without overflow. Data arrives from the accelerator as packed bits, and Python's numerical library has 8-, 16- and 32-bit integers but no 24-bit type, so the PYNQ

driver unpacked the 1024 values of every frame bit by bit in interpreted Python. That loop accounted for the 521.91 ms.

Build C removes the sigmoid from the network and applies it on the ARM processor instead. The network now ends with a convolution, batch normalization and a quantizer, which FINN converts entirely into thresholds, so the accelerator returns a finished 8-bit value per cell. Eight-bit integers are a native type, the driver reads them directly, and measured latency falls to 3.91 ms. Applying the sigmoid to 1024 values on the processor takes negligible time. Table 9 compares the two builds.

Measurement	Build B (24-bit output)	Build C (8-bit output)
Accelerator, frames back to back	0.74 ms	0.74 ms
Accelerator, single frame	3.00 ms	3.00 ms
Measured on the board	521.91 ms	3.91 ms

Table 9: The same accelerator before and after moving the sigmoid to the processor.

FINN reports two accelerator figures: 0.74 ms per frame when frames arrive continuously and the pipeline stays full, and 3.00 ms for one frame passing through an empty pipeline. A planner solves one problem at a time, so the second is the relevant one. The remaining 0.91 ms between 3.00 and 3.91 ms is data transfer and driver overhead on the ARM processor; a second session measured 4.05 ms, so the figure varies by a few percent between runs.

The change has a cost. Without the sigmoid, the network must reproduce the float encoder's values before the sigmoid, which range from about -9 to $+9$, rather than values between 0 and 1. That is harder: the training loss was 0.427 against 0.009, and p_{opt} fell from 0.750 to 0.655, while the reductions in expanded and generated nodes changed little.

I expected the difficulty to come from the extreme values, where the sigmoid is flat and differences barely change the guidance map. Build D tested this by clipping the training targets to ± 6 , beyond which the sigmoid is within 0.3% of 0 or 1; Table 10 gives the result.

Variant	Training loss (L1)	p_{opt}	p_{exp}	p_{gen}
B sigmoid on the accelerator	0.009	0.750	0.407	0.221
C sigmoid on the processor	0.427	0.655	0.412	0.228
D as C, targets clipped to ± 6	0.424	0.680	0.335	0.188

Table 10: Clipping the training targets does not recover the path quality lost by moving the sigmoid to the processor.

The hypothesis was wrong. Clipping barely changed the training loss, from 0.427 to 0.424, so the extreme values were not the source of the difficulty. p_{opt} improved slightly, to 0.680, but the reductions in expanded and generated nodes fell to 0.335 and 0.188. Build C was kept. Recovering build B's path quality with the sigmoid on the processor — for example with a loss that weights errors by how much they change the guidance value — remains open.

10.4 Comparison with software

A speed comparison is only meaningful when the same network runs on each processor. Table 11 gives the deployed 8/16/16 encoder in three places: on the accelerator, on the board's ARM processor in a numpy implementation with the convolutions as matrix products, and on the x86 server in PyTorch.

Platform	Network	Latency	vs FPGA
FPGA accelerator on the board (build C)	8/16/16	3.91 ms	—
ARM Cortex-A9 on the board, numpy	8/16/16	35.24 ms	9.0× slower
x86 server, PyTorch, 2 threads	8/16/16	0.74 ms	5.3× faster
x86 server, PyTorch, 2 threads	published 391k	12.89 ms	3.3× slower

Table 11: Single-frame encoder latency. The first three rows are the same network on different processors; the last is the published encoder, given for the system-level comparison in the text.

On the board, the accelerator computes the guidance map nine times faster than the ARM processor beside it, which is the comparison an embedded deployment faces: the planner runs on this board, and the choice is between the processor and the fabric. On a desktop x86 processor the result reverses. At 3,831 parameters the network is small enough that a processor clocked at several gigahertz with vector units finishes it in 0.74 ms, 5.3 times faster than the accelerator on the board. At the system level the comparison is between the published encoder, which a planner would otherwise run on a host processor, and the accelerated one on the board: 3.91 ms against the 6.70 ms of Section 4, a factor of 1.71, or 1.66 using the 4.05 ms measured in the second board session. The conservative figure is used throughout. That comparison is between different networks on different processors, and the smaller encoder also plans slightly worse: p_exp 0.412 against 0.441, and p_opt 0.655 against 0.800.

x86 timings on the shared server vary with the number of threads allocated: the published encoder measured 6.70 ms in the profiling of Section 4 and 12.89 ms in the two-thread session used for Table 11. The figures in Table 11 therefore all come from one session. The ARM figure is a straightforward numpy implementation rather than hand-optimized code, so a tuned implementation could narrow the ninefold gap.

The board measurement also changes where the time goes. A complete plan on the board takes 26.88 ms, the two columns of Table 7 added together: 5.00 ms to produce the guidance map and 21.88 ms for the search on the ARM processor. With the encoder accelerated, the search accounts for 81% of planning time — the reverse of the desktop profile in Section 4, where it was 8%. On an embedded platform, the hardware search engine of the original proposal, sized in Section 3, is therefore the natural next step.

11. LIMITATIONS

- All results use a single map family, 32×32 mazes from the Motion Planning dataset [4].
- Quantized encoders were trained by distillation from the float encoder rather than through the full Neural A* objective. The two aim at the same result but are not identical.
- All four builds use the same architecture. The other rows of Table 5 are projected from a fit on two measured designs, assuming LUT cost depends on MACs per cycle and not on channel width.
- Build C trades path optimality for a usable output format; clipping the training targets did not recover it, and other training approaches were not tried.
- Two further reductions in latency were identified but not attempted: a C driver in place of the Python one, which accounts for about 0.9 ms of the 3.91 ms, and moving the search to hardware (Section 10.4).
- x86 timings vary with thread allocation on the shared server, and the ARM baseline is a numpy implementation rather than hand-optimized code (Section 10.4).
- The encoder is the reference repository's convolutional network, not the U-Net [5] with a VGG-16 backbone used in [1], so quality figures are not directly comparable with the paper's.

12. CONCLUSION

Profiling moved the accelerator from the search to the encoder, which accounts for 92–95% of inference time, and showed that a Bloom filter offers nothing over an exact bit-vector on grids. The published encoder exceeds a Zynq-7020 by 3.47 times; a 3,831-parameter encoder trained by distillation fits in 66.3% of the device, meets timing, computes guidance maps nine times faster than the board's ARM processor running the same network, and plans on the board within 0.01 of the software model on every metric. A desktop x86 processor runs the same small network faster than the accelerator, so at this model size the accelerator's case is as an embedded one. On the board, with the encoder accelerated, the search on the ARM processor becomes 81% of planning time, which makes the hardware search engine of the original proposal the natural next step for an embedded platform. Two findings concern measurement rather than design: an output format that FINN's reports did not reveal made the deployed system 174 times slower than its accelerator until the sigmoid was moved to the processor, and the reduction in generated nodes — the work a hardware search performs — is about half the reduction in expanded nodes.

REFERENCES

- [1] R. Yonetani, T. Tanai, M. Barekatin, M. Nishimura, and A. Kanezaki. Path Planning using Neural A* Search. In Proceedings of the 38th International Conference on Machine Learning (ICML), 2021. arXiv:2009.07476.
- [2] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference. In Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA), pages 65-74, 2017.
- [3] W. Jiang, H. Hu, T. Hoefler, and G. Alonso. Fast Graph Vector Search via Hardware Acceleration and Delayed-Synchronization Traversal. Proceedings of the VLDB Endowment, 18(11):3797-3811, 2025.
- [4] M. Bhardwaj, S. Choudhury, and S. Scherer. Learning Heuristic Search via Imitation. In Proceedings of the Conference on Robot Learning (CoRL), 2017.
- [5] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. In MICCAI, pages 234-241, 2015.
- [6] P. E. Hart, N. J. Nilsson, and B. Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2):100-107, 1968.
- [7] B. H. Bloom. Space/Time Trade-offs in Hash Coding with Allowable Errors. Communications of the ACM, 13(7):422-426, 1970.
- [8] A. Pappalardo. Brevitas: Quantization-Aware Training in PyTorch. <https://github.com/Xilinx/brevitas>, 2023.